

Real-Time $p:q$ Deterministic Pushdown Automata:
 Generalization, Normalization, and Exact Control Width
A WRITE-SEARCH-CANCEL Theory for Weighted Binary Balance Languages

Alp Eren Bütün
 Independent Researcher, Türkiye
 github.com/alperenbutun

August 2026

Abstract

We study a completion-aware deterministic pushdown architecture for the weighted binary balance languages

$$L_{p,q} = \{w \in \{0,1\}^* : qN_1(w) = pN_0(w)\}, \quad \gcd(p,q) = 1.$$

The starting point is an explicit 2:1 center/left/right DPDA whose state and stack jointly encode what is still required to restore balance. We extract from that machine a general shortest-demand semantics. The primitive zero-weight kernel is $K = (q,p)$; mismatches are handled by moving a one-cell LIFO blocker into finite control as signed *search debt*; and a canonical WRITE operation returns the distributed state-stack representation to a shortest-demand stack normal form.

For every coprime positive p,q we define a centered real-time family with debt states

$$D_{-h}, \dots, D_{-1}, D_0, D_1, \dots, D_h, \quad h = \left\lfloor \frac{p+q-1}{2} \right\rfloor,$$

using only the work symbols 0,1 and a fixed bottom marker Z_0 that is preserved throughout binary-input processing. We prove a residual invariant, an exact virtual-demand projection onto the unique minimum completion demand, a canonical-segment invariant implying two-cell qualitative stack visibility, exact WRITE normalization, and zero-residual correctness. Within a precisely stated canonical deferred-SEARCH realization model, the same construction has exact control width

$$2h+1 = \begin{cases} p+q, & p+q \text{ odd,} \\ p+q-1, & p+q \text{ even.} \end{cases}$$

Finally, specializing the general theory back to $(p,q) = (2,1)$ recovers the original $C/L/R$ skeleton, the potentials 0, +3, -3, and the characteristic replacement words. The only remaining operational difference is a normalization schedule: the historical machine eagerly materializes two right-state debts, whereas the centered specialization retains them lazily in finite control. Both realizations have the same shortest-demand semantics.

1 Introduction

For coprime positive integers p,q , consider the weighted binary balance language

$$L_{p,q} = \{w \in \{0,1\}^* : qN_1(w) = pN_0(w)\}.$$

Recognition of such a language is not, by itself, the difficult point. Deterministic one-counter computation is classical, and a signed count difference can serve as a recognition coordinate [1]. The question pursued here is more structural:

Can a deterministic pushdown machine maintain a *literal completion demand* on its stack, keep both demand types locally visible, repair LIFO mismatches with bounded finite control, and do so uniformly for every coprime ratio $p:q$?

The question arose from a concrete 2:1 DPDA developed by the author in 2023. That machine has a center state C , two side states L and R , a binary work stack, a visible cancellation transition, and end-of-input completion rules. Its unusual feature is not merely that it recognizes

$$L_{2,1} = \{w : N_1(w) = 2N_0(w)\},$$

but that its state and stack jointly describe what remains to be supplied if the processed prefix is to be completed to the language.

Three mechanisms in that design initially look unrelated:

WRITE	SEARCH	CANCEL.
-------	--------	---------

WRITE creates outstanding completion needs. SEARCH resolves the fact that a needed symbol can sit behind a one-cell LIFO blocker. CANCEL removes a matched need online. The central observation of this paper is that these are three operational views of one arithmetic object: the unique minimum nonnegative completion demand.

The generalization developed below is deliberately richer than a compact residual counter. A compact one-counter or residue-in-state implementation can recognize $L_{p,q}$ without preserving a literal list of outstanding needs. Here we instead ask the stack to retain that literal semantic role while finite control carries the temporary debt created by local LIFO search. This distinction is important throughout the paper.

Main contributions. The paper establishes the following chain.

1. Every residual has a unique minimum nonnegative completion demand, and all other nonnegative demands differ from it by copies of the primitive kernel $K = (q, p)$.
2. A mismatch between an input symbol and the visible demand cell produces a signed one-cell debt. Moving that blocker into finite control leads to a centered debt coordinate with potential increments of $p + q$.
3. A minimal kernel-carry equation determines the amount written whenever distributed state debt is materialized back into the stack.
4. The resulting centered family is a real-time DPDA for every coprime p, q . Between WRITES it performs only pops, and every reachable stack is a suffix of a canonical shortest-demand word. Consequently, if both work symbols occur anywhere in the stack, both occur among the first two cells.
5. In a canonical deferred-SEARCH realization model, the finite-control debt width is exact: $2\lfloor(p + q - 1)/2\rfloor + 1$ states are necessary and sufficient.
6. Specializing the general construction to 2:1 recovers the original $C/L/R$ architecture. The historical and centered versions differ only in when certain right-state debts are materialized: eager and lazy normalization schedules have the same shortest-demand projection.

The proof architecture therefore forms a loop rather than a one-way extrapolation:

$$\boxed{\text{2:1 machine}} \longrightarrow \boxed{p:q \text{ theory}} \longrightarrow \boxed{\text{exact deferred control width}} \longrightarrow \boxed{\text{2:1 recovery}}.$$

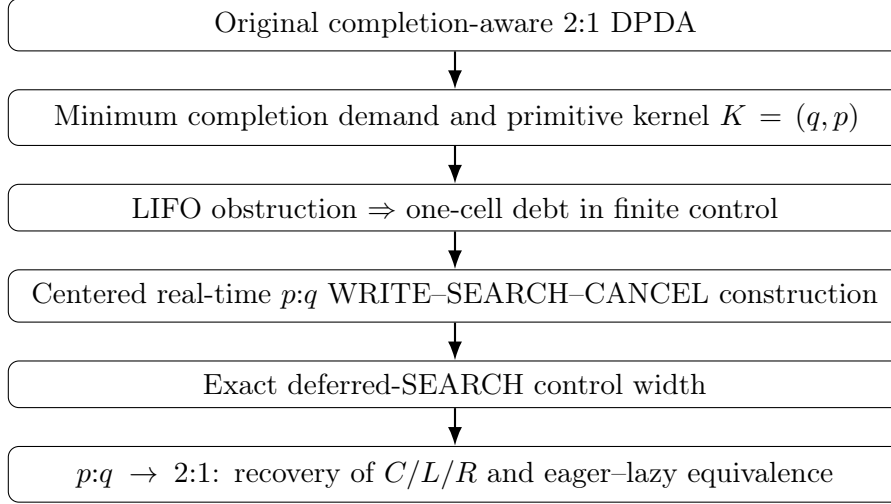


Figure 1: Conceptual route of the paper. The general theory is required to explain the original 2:1 architecture when specialized back to the motivating ratio.

2 The motivating completion-aware 2:1 DPDA

We begin with the standard empty-word-included presentation of the motivating design. A transition label

$$a, A/\gamma$$

means that input a is consumed, the top stack symbol A is replaced by the word γ , and the indicated state transition is taken. The leftmost work symbol is the stack top. The symbol Z_0 is the bottom marker.

The dashed callouts in figure 2 are not ordinary ε -moves. They are completion operations enabled only when the right endmarker $\dashv$ is the next unread symbol. This keeps post-input completion separate from ordinary recognition and preserves deterministic transition semantics.

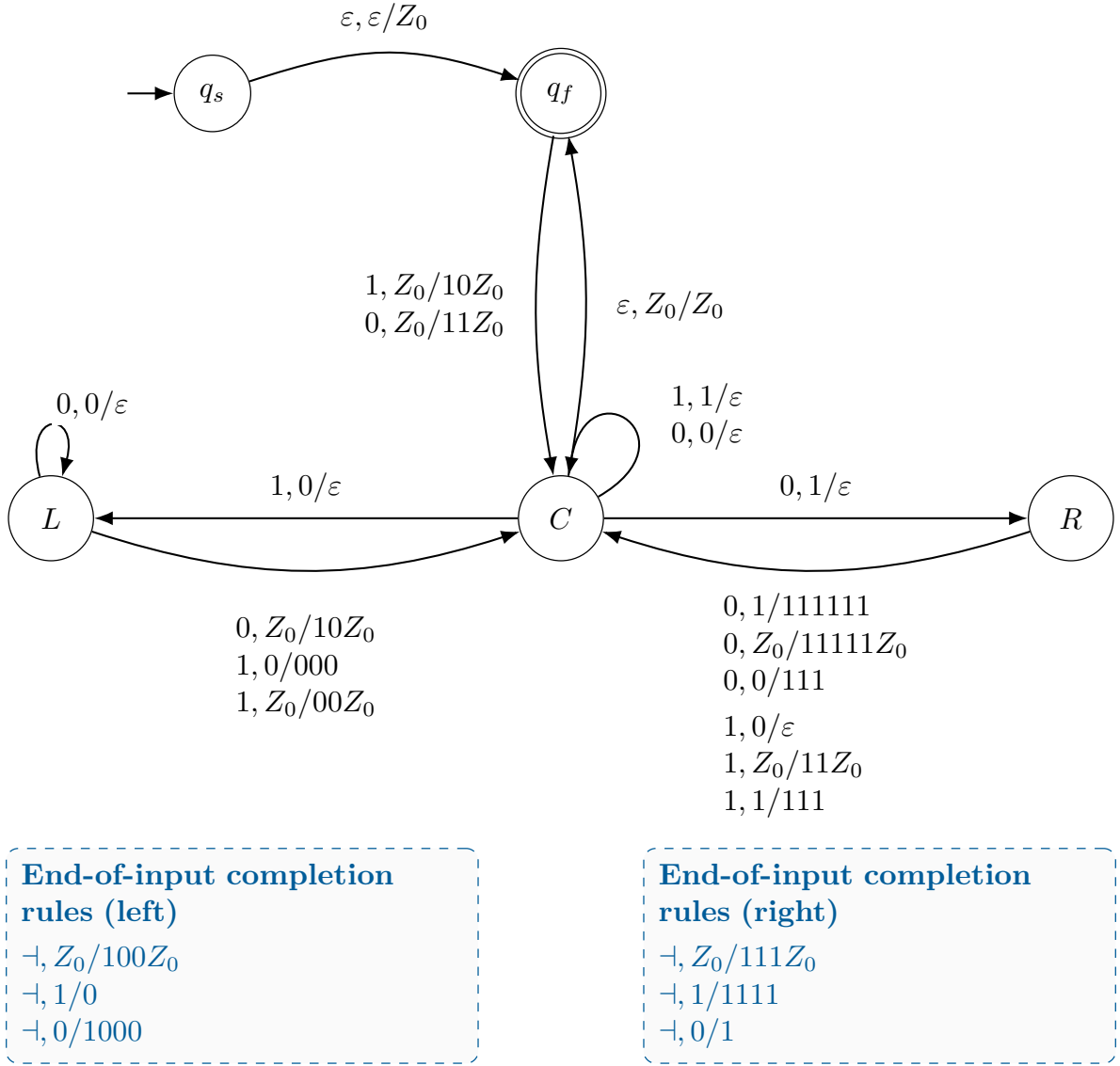


Figure 2: Formal redrawing of the original completion-aware 2:1 DPDA with the empty word included. Solid arcs process binary input. Dashed boxes list only right-endmarker completion operations. The center/left/right core is the object generalized later in the paper.

Projecting away the initialization/acceptance wrapper gives the semantic core $\{C, L, R\}$. If the physical stack is SZ_0 , write $S \in \{0, 1\}^*$ for its logical stack. The reachable core stores satisfy

$$\mathcal{S}_C = 0^* \cup 1^* \cup \{10\}, \quad \mathcal{S}_L = 0^*, \quad \mathcal{S}_R = 1^* \cup \{0\}.$$

In particular, the left-state transition $0, 0 / \varepsilon$ is a genuine online cancellation, while a reachable left configuration never has top symbol 1.

For the 2:1 core set

$$D(S) = 2N_0(S) - N_1(S), \quad \Phi(C) = 0, \quad \Phi(L) = 3, \quad \Phi(R) = -3.$$

Then every processed prefix satisfies

$$\rho_{2,1} = D(S) + \Phi(q_{\text{state}}), \tag{1}$$

where $\rho_{2,1} = N_1 - 2N_0$ is the signed input residual. Thus an empty logical stack in a side state does not mean zero residual: finite control itself carries part of the completion demand.

Example 2.1 (The prefix 111). Ignoring Z_0 in the logical notation,

$$(C, 111 \neg, \varepsilon) \vdash (C, 11 \neg, 10) \vdash (C, 1 \neg, 0) \vdash (L, \neg, \varepsilon).$$

The processed prefix has residual $\rho_{2,1}(111) = 3$. The right-endmarker completion rule $\neg, Z_0/100Z_0$ materializes the missing state contribution and produces the shortest completion 100. Indeed, every completion containing x zeros and y ones must satisfy $2x - y = 3$, whose unique minimum nonnegative solution is $(2, 1)$.

The purpose of the remaining sections is to explain this behavior without relying on the accidental constants of the ratio 2:1.

3 Residuals, minimum completion demand, and the primitive kernel

Fix coprime $p, q \geq 1$ and put

$$m = p + q.$$

For a prefix w define

$$\rho(w) = qN_1(w) - pN_0(w). \quad (2)$$

If a continuation contains x zeros and y ones, then it completes w exactly when

$$px - qy = \rho(w). \quad (3)$$

For a demand vector $v = (x, y) \in \mathbb{N}_0^2$, write

$$W(v) = px - qy.$$

Theorem 3.1 (Unique minimum completion demand). *For every $r \in \mathbb{Z}$ there is a unique nonnegative solution*

$$\mu(r) = (x_r, y_r)$$

of $px - qy = r$ minimizing $x + y$. If (x, y) is any nonnegative solution, then

$$(x, y) = \mu(r) + k(q, p)$$

for a unique integer $k \geq 0$.

Proof. If (x, y) and (x', y') have the same weight, then

$$p(x - x') = q(y - y').$$

Since $\gcd(p, q) = 1$, there is an integer k with

$$x - x' = qk, \quad y - y' = pk.$$

Thus all integer solutions lie on one affine line in the primitive direction

$$K = (q, p).$$

Among the nonnegative points on this line there is a unique first point in the K -direction. Adding one copy of K increases $x + y$ by $p + q = m$, so that first point is the unique length-minimizing nonnegative solution. $\square$

The vector

$$\boxed{K = (q, p)} \quad (4)$$

is the primitive zero-weight kernel because $W(K) = pq - qp = 0$.

Definition 3.2 (Kernel reduction). For $(x, y) \in \mathbb{N}_0^2$ define

$$\text{red}_{p,q}(x, y) = (x - kq, y - kp),$$

where

$$k = \min\left(\left\lfloor \frac{x}{q} \right\rfloor, \left\lfloor \frac{y}{p} \right\rfloor\right).$$

Corollary 3.3 (Kernel reduction). For every nonnegative (x, y) ,

$$\boxed{\text{red}_{p,q}(x, y) = \mu(W(x, y)).}$$

Online demand update. If the current shortest demand is $(x, y) = \mu(r)$, then reading a new input symbol changes the residual by $-p$ on input 0 and by $+q$ on input 1. The new minimum demand is

$$T_0(x, y) = \begin{cases} (x - 1, y), & x > 0, \\ (q - 1, y + p), & x = 0, \end{cases} \quad (5)$$

and

$$T_1(x, y) = \begin{cases} (x, y - 1), & y > 0, \\ (x + q, p - 1), & y = 0. \end{cases} \quad (6)$$

The first branch is literal matching and cancellation. The second says that, if no matching need exists, the incoming symbol supplies one member of a primitive balanced kernel and the remaining members of that kernel become the new demand.

4 Canonical ordering and the LIFO obstruction

The vector $\mu(r)$ determines the numbers of required zeros and ones, but not their order. For the pushdown implementation we use the canonical two-visible ordering

$$\eta(x, y) = \begin{cases} (10)^y 0^{x-y}, & x \geq y, \\ (10)^x 1^{y-x}, & y > x. \end{cases} \quad (7)$$

If both symbols occur, the word begins with 10; every suffix that remains mixed begins with either 10 or 01.

A static ordering is not, however, enough for an online DPDA. The obstruction is already visible at 3:2:

$$S = 1010.$$

Both symbols are visible in the first two cells. If input 0 is intended to remove the second cell and a naive search temporarily pops the top 1, deletes the 0, and immediately restores the blocker, then

$$1010 \longrightarrow 110,$$

and the surviving 0 is now at depth three.

Lemma 4.1 (Static two-visible obstruction). *Suppose a mixed binary stack contains at least two zeros and at least two ones and begins with 10. A purely static strategy that requires deletion of either visible symbol type to leave a mixed suffix with both symbol types still in the first two cells cannot succeed for both deletion directions by restoring a displaced first-cell blocker immediately.*

Proof. If deleting the first 1 must leave a mixed two-visible stack, then the next two surviving symbols must be different, so the third original symbol must be 1. If instead the second-cell 0 is deleted and the first 1 is immediately restored, preserving two-visibility requires the next surviving symbol under that restored 1 to be 0, so the third original symbol must be 0. The same third cell cannot satisfy both requirements. $\square$

The repair is therefore not a deep stack scan. It is a one-cell transfer of information:

$$\boxed{\text{SEARCH} = \text{displace one blocker into finite control.}}$$

The blocker is not immediately restored to the stack. Its signed effect is carried as finite-control debt until a later WRITE normalizes the representation.

5 Debt states and virtual demand

A one-cell mismatch changes the relationship between physical stack content and semantic demand by a multiple of the kernel length $m = p + q$. This motivates a centered integer debt coordinate.

For $d \in \mathbb{Z}$ define the virtual demand

$$\boxed{V_d = \mu(dm)}. \quad (8)$$

Its weight is

$$W(V_d) = dm.$$

A state D_d therefore represents a finite-control potential

$$\Phi(D_d) = dm. \quad (9)$$

For a logical stack S , let

$$P(S) = (N_0(S), N_1(S)).$$

Definition 5.1 (Virtual-demand projection). For a configuration (D_d, S) define

$$\boxed{\Theta(D_d, S) = \text{red}_{p,q}(P(S) + V_d)}. \quad (10)$$

Theorem 5.2 (Virtual-demand projection). *If a configuration satisfies the residual invariant*

$$\rho = W(P(S)) + dm, \quad (11)$$

then

$$\boxed{\Theta(D_d, S) = \mu(\rho)}.$$

Proof. The nonnegative vector $P(S) + V_d$ has weight

$$W(P(S)) + W(V_d) = W(P(S)) + dm = \rho.$$

By theorem 3.3, kernel reduction of any nonnegative vector of weight ρ is exactly the unique minimum demand $\mu(\rho)$. $\square$

Thus the rich machine may store more information than the residual, but its semantic projection is rigid: every reachable configuration denotes the same unique shortest demand as its residual.

6 The minimal kernel-carry WRITE equation

The arithmetic of WRITE can be separated from the question of when WRITE is invoked. Let

$$e_0 = (1, 0), \quad e_1 = (0, 1).$$

Suppose a transition consumes input a , removes visible top symbol z , changes state from D_s to D_t , and writes a replacement word u in place of z . Ignoring the unchanged suffix, preservation of the residual invariant forces

$$P(u) = e_z - e_a + V_s - V_t + \kappa K \quad (12)$$

for some integer κ . At the bottom marker the term e_z is omitted.

Put

$$B = e_z - e_a + V_s - V_t.$$

The smallest kernel carry making both coordinates nonnegative is

$$\kappa_* = \max \left(\left\lceil -\frac{B_0}{q} \right\rceil, \left\lceil -\frac{B_1}{p} \right\rceil \right). \quad (13)$$

Theorem 6.1 (Minimal kernel-carry rewrite). *Among all nonnegative replacement vectors compatible with the source state, target state, input symbol, visible top symbol, and residual invariant, the unique shortest vector is*

$$R_* = B + \kappa_* K. \quad (14)$$

Every other nonnegative solution is $R_* + \ell K$ for some $\ell \geq 1$ and is longer by $\ell(p + q)$ symbols.

Proof. All invariant-preserving solutions differ by an integer multiple of K by theorem 3.1. The value κ_* is exactly the first integer shift for which both coordinates are nonnegative. Each further shift adds $q + p$ symbols. $\square$

When a concrete word is required, we use the canonical ordering $\eta(R_*)$. This explains the large replacement strings that appear in the original 2:1 table: their lengths are forced by residual arithmetic plus the decision to discharge finite-control debt into a top-only pushdown rewrite.

7 The centered real-time $p:q$ construction

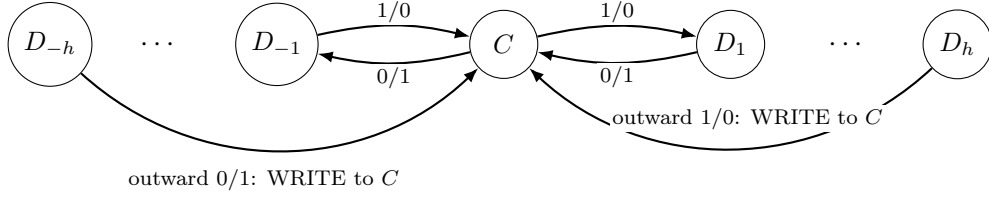
Set

$$h = \left\lfloor \frac{m-1}{2} \right\rfloor. \quad (15)$$

The centered semantic core is

$$Q = \{D_{-h}, \dots, D_{-1}, D_0, D_1, \dots, D_h\}, \quad (16)$$

with $D_0 = C$. The stack alphabet is $\{0, 1, Z_0\}$, where Z_0 is fixed throughout binary-input processing and is removed only by the final right-endmarker acceptance move.



The adjacent mismatch rules repeat along the entire rail.
At every D_d , matching 0/0 or 1/1 simply pops and keeps d .

Figure 3: Schematic centered debt rail. Labels 1/0 and 0/1 mean input/top mismatch, not stack replacement notation. Interior mismatch steps pop the visible cell and move debt by one; an outward mismatch at $\pm h$ triggers WRITE back to the center.

We now give the exact transition schema. A logical stack is written with top at the left.

MATCH. For $z \in \{0, 1\}$ and every $|d| \leq h$,

$$(D_d, zT) \xrightarrow{z} (D_d, T). \quad (17)$$

Interior SEARCH. For $d < h$,

$$(D_d, 0T) \xrightarrow{1} (D_{d+1}, T), \quad (18)$$

and for $d > -h$,

$$(D_d, 1T) \xrightarrow{0} (D_{d-1}, T). \quad (19)$$

The displaced blocker is not written back; its signed effect is recorded by d .

Outer-boundary WRITE. Put

$$U_+ = \eta(\mu((h+1)m)), \quad U_- = \eta(\mu(-(h+1)m)).$$

Then

$$(D_h, 0T) \xrightarrow{1} (C, U_+T), \quad (20)$$

and

$$(D_{-h}, 1T) \xrightarrow{0} (C, U_-T). \quad (21)$$

Bottom WRITE. For every $|d| \leq h$,

$$(D_d, Z_0) \xrightarrow{1} (C, \eta(\mu(dm + q))Z_0), \quad (22)$$

$$(D_d, Z_0) \xrightarrow{0} (C, \eta(\mu(dm - p))Z_0). \quad (23)$$

Acceptance. A mandatory right endmarker is permitted only at the balanced empty configuration:

$$(C, Z_0) \xrightarrow{\perp} (C, \varepsilon), \quad (24)$$

with no right-endmarker rule from any other reachable configuration. Acceptance is by empty stack after the endmarker.

Every binary transition consumes exactly one input symbol; there are no ε -moves in the centered construction.

Theorem 7.1 (Residual invariant). *After every processed binary prefix, if the centered configuration is (D_d, SZ_0) , then*

$$\boxed{\rho = W(P(S)) + dm.} \quad (25)$$

Proof. The initial configuration (C, Z_0) has both sides zero. A matching pop of 0 decreases $W(P(S))$ by p , matching the input-0 change of ρ ; a matching pop of 1 increases $W(P(S))$ by q . A mismatch 1/0 pops one zero, changing stack weight by $-p$, and increases the state potential by $m = p + q$, for net change q . The mismatch 0/1 is symmetric, with net change $-p$. At the positive boundary, $W(U_+) = (h + 1)m$, so replacing a top zero and resetting the state from h to 0 gives the same net $+q$ update. The negative boundary is symmetric. Bottom WRITE is defined directly with target weight $dm + q$ or $dm - p$. Hence the invariant is preserved by every binary transition. $\square$

8 Canonical-segment invariant and two-cell visibility

The key proof simplification is that the machine does not need a large regular-language description of every reachable store. It is enough to track what happens between WRITES.

Definition 8.1 (Minority size). For a demand vector (x, y) put

$$\text{minc}(x, y) = \min(x, y).$$

Lemma 8.2 (WRITE-frontier minority bound). *Let $h = \lfloor (m - 1)/2 \rfloor$.*

(a) *For every $|d| \leq h$, each of*

$$\mu(dm + q), \quad \mu(dm - p)$$

has minority size at most h .

(b) *If*

$$\mu((h + 1)m) = (x_+, y_+),$$

then $y_+ \leq h < x_+$. If

$$\mu(-(h + 1)m) = (x_-, y_-),$$

then $x_- \leq h < y_-$.

Proof. For part (a), note that both bottom-WRITE residuals have the form

$$r = jm + q, \quad -h - 1 \leq j \leq h,$$

because $dm - p = (d - 1)m + q$. Let $\mu(r) = (x, y)$ and $\ell = x + y$. Modulo m , since $p \equiv -q$, equation $px - qy = r \equiv q$ gives

$$-q\ell \equiv q \pmod{m}.$$

As $\gcd(q, m) = 1$, $\ell \equiv -1 \pmod{m}$, so $\ell = km - 1$ for some $k \geq 1$.

Assume for contradiction that $x, y \geq h + 1$. Then $\ell \geq 2h + 2 > m - 1$, hence $k \geq 2$. Since $\mu(r)$ is kernel-free, either $x < q$ or $y < p$. If $x < q$, then $x \geq h + 1$ forces $q \geq h + 2$, while

$$r = mx - q\ell = m(x - qk) + q$$

gives $j = x - qk \leq -q - 1 \leq -h - 3$, contradicting $j \geq -h - 1$. If $y < p$, then $p \geq h + 2$, while

$$r = p\ell - my = m(pk - y - 1) + q$$

gives $j = pk - y - 1 \geq p \geq h + 2$, contradicting $j \leq h$. Thus $\min(x, y) \leq h$.

For part (b), write $D = h + 1$. For positive residual Dm , every integer solution has

$$x = D + qk, \quad y = pk - D.$$

The minimum nonnegative solution uses $k = \lceil D/p \rceil$, hence $0 \leq y < p$. If $p \leq D$, then $y \leq p - 1 \leq h$. If $p > D$, then $k = 1$ and

$$y = p - D \leq (m - 1) - D \leq h.$$

Also $x \geq D = h + 1$. The negative case is symmetric. $\square$

Lemma 8.3 (Boundary guard). *Suppose that since the most recent WRITE the current stack is a suffix of a canonical word $\eta(x, y)$ with $\min(x, y) \leq h$.*

- (a) *If the current state is D_h and the stack begins with 0, then an outward 1/0 mismatch can occur only on a unary zero suffix 0^{t+1} .*
- (b) *If the current state is D_{-h} and the stack begins with 1, then an outward 0/1 mismatch can occur only on a unary one suffix 1^{t+1} .*

Proof. Consider the positive case. Since the last WRITE, debt starts at zero. Reaching debt h requires at least h positive mismatch events, and each such event pops a zero from the pre-existing canonical stack. Thus at least h zeros have already been popped.

If the surviving suffix begins with 0 and still contains a 1, then this current zero lies in the alternating part of $\eta(x, y)$ rather than in its final unary tail. Every previously popped zero also lies before this position in the alternating part and therefore has a distinct preceding one that has already been popped. Together with the current zero and a later surviving one, the original canonical word must contain at least $h + 1$ zeros and at least $h + 1$ ones. This contradicts $\min(x, y) \leq h$. Hence the suffix is unary zero. The negative case is symmetric. $\square$

Theorem 8.4 (Canonical-segment invariant). *Immediately after every WRITE, the centered machine is in the center state with a stack*

$$\eta(\mu(r))Z_0$$

for its current residual r , and the corresponding minimum demand has minority size at most h . Between WRITES, the logical stack is a suffix of that canonical word.

Proof. We argue by induction over successive WRITE episodes. Initially the logical stack is empty. A bottom WRITE has exactly the displayed canonical form by definition, and theorem 8.2(a) gives the minority bound. Between two successive WRITES, MATCH and interior SEARCH only pop the first work symbol, so the stack remains a suffix of the canonical word created by the preceding WRITE.

For a positive outer-boundary WRITE, theorem 8.3 gives a source stack 0^{t+1} . Let

$$\mu((h + 1)m) = (x, y).$$

By theorem 8.2(b), $y \leq h < x$, so $\eta(x, y)$ ends in 0. The transition produces

$$\eta(x, y)0^t = \eta(x + t, y).$$

Its weight is $(h + 1)m + pt$, exactly the new residual by theorem 7.1. Moreover $y < p$ from the proof of theorem 8.2, so $(x + t, y)$ is kernel-free; hence it is the minimum demand for the new residual. Its minority size remains $y \leq h$. The negative boundary is symmetric, appending a unary one tail to a canonical word whose minority zero count is at most h . $\square$

Corollary 8.5 (Two-cell qualitative visibility). *For every reachable logical stack S ,*

$$\boxed{\text{alph}(S) = \text{alph}(\text{pref}_2(S))}. \quad (26)$$

Equivalently, if both 0 and 1 occur anywhere in S , then both occur among its first two symbols.

Proof. Every canonical word $\eta(x, y)$, and every suffix of it, avoids the factors 001 and 110. Therefore a suffix that is still mixed cannot begin with two equal symbols: it begins with 10 or 01. Apply theorem 8.4. $\square$

The significance of Corollary 8.5 is qualitative rather than quantitative. The first two cells reveal which demand types are present, not the unbounded magnitudes of those demands.

9 Correctness and real-time normalization

The preceding section already proves the strongest form of WRITE correctness: WRITE does not merely preserve the residual; it returns the physical stack to the exact minimum-demand normal form.

Theorem 9.1 (Exact WRITE normalization). *After every bottom or outer-boundary WRITE, the centered configuration is*

$$(C, \eta(\mu(\rho))Z_0),$$

where ρ is the residual after the consumed input symbol.

Proof. This is the first assertion of theorem 8.4. $\square$

Theorem 9.2 (Zero-residual correctness). *At the end of any binary prefix,*

$$\boxed{\rho = 0 \iff (D_d, S) = (C, \varepsilon)}. \quad (27)$$

Proof. The reverse implication follows immediately from the invariant. For the empty prefix the forward implication is immediate from the initial configuration (C, Z_0) , whose logical stack is empty. Now consider a nonempty processed prefix. Its first binary symbol is handled by a bottom WRITE, so a most recent WRITE exists before the current configuration. Immediately after that WRITE the machine was at

$$(C, \eta(\mu(r_0)))$$

for some residual r_0 . Write $\mu(r_0) = (x, y)$. Since then, and before any further WRITE, each input symbol has popped exactly one pre-existing demand cell. Suppose k cells have been popped and the intervening input segment contains u zeros and v ones. Then

$$k = u + v \leq x + y.$$

If the current residual is zero, that input segment completes r_0 , so

$$pu - qv = r_0.$$

By minimality of $\mu(r_0)$, every such nonnegative completion has length at least $x + y$. Hence

$$u + v = x + y,$$

so the entire canonical stack has been consumed and $S = \varepsilon$. The residual invariant then gives $dm = 0$, hence $d = 0$ and the state is C . $\square$

Theorem 9.3 (Real-time centered rich DPDA). *For every coprime $p, q \geq 1$, the transition schema in section 7 defines a deterministic ε -free binary-input pushdown machine with*

$$|Q| = 2 \left\lfloor \frac{p+q-1}{2} \right\rfloor + 1$$

semantic core states and stack alphabet $\{0, 1, Z_0\}$. With the mandatory right endmarker of equation (24), it recognizes $L_{p,q}$. Its reachable stacks satisfy two-cell qualitative visibility, and every WRITE returns to the exact shortest-demand normal form.

Proof. For every state, binary input symbol, and visible top category (0, 1, or Z_0), exactly one of MATCH, interior SEARCH, boundary WRITE, or bottom WRITE applies. Every binary move consumes one symbol and no ε -transition is used. The residual invariant is theorem 7.1; the visibility and WRITE claims are theorems 8.5 and 9.1. By theorem 9.2, the right endmarker is enabled exactly when the binary input has zero residual, which is exactly membership in $L_{p,q}$. $\square$

The state count may be written

$$|Q| = \begin{cases} p+q, & p+q \text{ odd,} \\ p+q-1, & p+q \text{ even.} \end{cases} \quad (28)$$

This is the exact size of the centered construction. It is not yet an unrestricted lower bound for all DPDAs recognizing $L_{p,q}$.

(p, q)	$m = p + q$	h	centered core states
(2, 1)	3	1	3
(3, 1)	4	1	3
(3, 2)	5	2	5
(4, 1)	5	2	5
(5, 2)	7	3	7

Table 1: Examples of the centered control width.

10 Rich-to-residual semantic projection

The centered machine retains physical information that a compact residual counter would discard. Nevertheless, theorem 5.2 gives an exact quotient map at every reachable configuration:

$$(D_d, S) \xrightarrow{\Theta} \mu(\rho) \xrightarrow{W} \rho.$$

Corollary 10.1 (Shortest-demand quotient). *Two reachable centered configurations with the same residual have the same Θ -image. Thus the residual transition graph is a semantic quotient of the rich configuration graph under the equivalence*

$$c_1 \sim_{\Theta} c_2 \iff \Theta(c_1) = \Theta(c_2).$$

The fibers of this quotient can retain normalization chronology: two configurations may represent the same shortest demand while storing different portions of it physically in the stack and virtually in finite control. This is exactly what will distinguish eager and lazy 2:1 schedules in section 13.

11 Exact control width in the canonical deferred-SEARCH model

An unrestricted state lower bound for every DPDA recognizing $L_{p,q}$ is not claimed. The following result concerns the operational model that motivated the centered construction.

Definition 11.1 (Canonical deferred-SEARCH realization). A canonical deferred-SEARCH realization is a literal-demand pushdown realization with the following interface.

1. A normalized demand vector $\mu(r)$ is represented by the canonical physical word $\eta(\mu(r))$.
2. During a SEARCH episode through that pre-existing canonical word, each binary step that consumes a pre-existing demand cell removes exactly one such cell. A mismatch is represented only in finite control, and no auxiliary debt marker is inserted into the remaining pre-existing word.
3. For the witness episode used below, every transition that consumes a pre-existing demand cell is a pure pop with respect to the work stack. In particular, the transition consuming the final pre-existing cell performs no materializing push, introduces no auxiliary work symbol, and leaves the fixed bottom marker Z_0 unchanged. Materialization may occur only on a subsequent transition.

The third condition is the *deferred* part of the model. It deliberately excludes eager schedules that materialize debt before, or on the same transition as, exhaustion of the witness block. The theorem below is exact for this model, not for arbitrary equivalent DPDAs. Continuation distinguishability is not assumed as part of the model; it follows from the language itself by the next lemma.

Lemma 11.2 (Residual distinguishability). *Let two processed prefixes have distinct residuals $r_1 \neq r_2$. Then they are distinguished by a common binary continuation followed by the mandatory right endmarker.*

Proof. Choose a binary word z whose Parikh vector is $\mu(r_1)$. Since $pN_0(z) - qN_1(z) = r_1$, appending z changes residual by $-r_1$. Thus the first prefix reaches residual zero, while the second reaches residual $r_2 - r_1 \neq 0$. The right endmarker must therefore be accepted after the first continuation and rejected after the second. $\square$

Let

$$h = \left\lfloor \frac{m-1}{2} \right\rfloor$$

and define the witness demand

$$\nu_h = \begin{cases} (h, h), & m \text{ odd,} \\ (h, h+1), & m \text{ even.} \end{cases} \quad (29)$$

Its canonical word is $(10)^h$ in the odd case and $(10)^h 1$ in the even case.

Lemma 11.3 (Canonical witness). *The vector ν_h is kernel-free and therefore*

$$\nu_h = \mu(W(\nu_h)).$$

It contains at least h zeros and at least h ones.

Proof. If m is odd and both coordinates of (h, h) allowed subtraction of $K = (q, p)$, then $h \geq p$ and $h \geq q$, giving $m = p + q \leq 2h = m - 1$, impossible. If m is even, subtraction of K from $(h, h+1)$ would require $q \leq h$ and $p \leq h+1$, hence $m \leq 2h+1 = m-1$, again impossible. Thus the displayed vector is the minimum nonnegative representative of its weight. $\square$

Lemma 11.4 (Debt distinguishability). *Starting from the canonical witness word $\eta(\nu_h)$, after the entire pre-existing word has been consumed by a deferred SEARCH episode, every debt value*

$$d \in \{-h, -h+1, \dots, h\}$$

can be realized while the physical pre-existing stack is empty. The resulting residual is dm .

Proof. Choose $d \geq 0$. On exactly d of the witness zeros, read input 1 instead of matching 0; match every other cell. The debt increases monotonically from 0 to d . For $d < 0$, mismatch exactly $|d|$ witness ones with input 0 and match every other cell. Because the witness contains at least h copies of each symbol, all values $-h \leq d \leq h$ are possible.

Let the witness contain x zeros and y ones. If A zero-cells are mismatched by input 1 and B one-cells are mismatched by input 0, then $d = A - B$. The consumed input has

$$u = x - d \text{ zeros}, \quad v = y + d \text{ ones}.$$

Starting from residual $W(x, y) = px - qy$, the final residual is

$$px - qy + q(y + d) - p(x - d) = d(p + q) = dm.$$

□

Theorem 11.5 (Exact deferred-SEARCH control width). *Every canonical deferred-SEARCH realization in the sense of Definition 11.1 requires at least*

$$\boxed{2h + 1}$$

finite-control states to distinguish the witness debts. The centered construction uses exactly $2h + 1$ debt states. Hence its control width is exact within this model:

$$\boxed{|Q|_{\min}^{\text{deferred}} = 2 \left\lfloor \frac{p + q - 1}{2} \right\rfloor + 1 = \begin{cases} p + q, & p + q \text{ odd,} \\ p + q - 1, & p + q \text{ even.} \end{cases}} \quad (30)$$

Proof. By Lemma 11.4, after the same canonical witness block has been exhausted there are $2h + 1$ possible residuals

$$-hm, \dots, -m, 0, m, \dots, hm.$$

By Definition 11.1, the transition consuming the final witness cell performs no materializing push, so in every case the work stack is empty and the physical stack consists only of the same bottom marker Z_0 . If two distinct residuals shared the same finite-control state, the two instantaneous configurations would therefore be physically identical. But Lemma 11.2 gives a common continuation that must distinguish the two residuals. Determinism from identical configurations would force identical behavior on that continuation, a contradiction. Thus all $2h + 1$ debts require distinct control states. The matching upper bound is the centered state set of equation (16). □

Remark 11.6 (Scope). Theorem 11.5 is intentionally architecture-relative. It does not rule out a smaller DPDA that abandons literal-demand storage, uses a different stack encoding, materializes debt eagerly, or otherwise leaves the deferred-SEARCH interface. Descriptive-complexity results for DPDAs are highly representation-sensitive, so the model boundary is part of the theorem statement rather than a technical footnote.

12 Specialization back to 2:1

The strongest test of the generalization is whether it explains the machine from which it arose. Put

$$p = 2, \quad q = 1, \quad m = 3, \quad h = 1.$$

Then the centered states are

$$D_{-1}, D_0, D_1.$$

Identify

$$\boxed{D_{-1} = R, \quad D_0 = C, \quad D_1 = L.} \quad (31)$$

Their potentials are

$$\boxed{-3, 0, +3,}$$

which is exactly the original $R/C/L$ potential pattern in equation (1).

The virtual demands are

$$V_C = (0, 0), \quad V_L = \mu(3) = (2, 1), \quad V_R = \mu(-3) = (0, 3).$$

The general transition schema now reproduces the characteristic historical replacements:

source situation	centered specialization	historical core
C , bottom, input 1	10	10
C , bottom, input 0	11	11
C , top 0, input 1	pop to L	pop to L
C , top 1, input 0	pop to R	pop to R
L , top 0, input 0	pop, stay L	pop, stay L
L , top 0, input 1	000, to C	000, to C
R , top 1, input 0	11111, to C	11111, to C
R , top 0, input 1	pop to C	pop to C

The bottom rules from L and R similarly recover 10, 00, 11111, and 11.

Theorem 12.1 (2:1 specialization and recovery). *Specializing the centered $p:q$ construction to $(p, q) = (2, 1)$ recovers:*

- (i) *the three-state $C/L/R$ semantic skeleton;*
- (ii) *the state potentials $0, +3, -3$;*
- (iii) *the center mismatch directions;*
- (iv) *the left cancellation rail;*
- (v) *the right locality-repair rail; and*
- (vi) *the arithmetic of the historical WRITE words 10, 11, 00, 000, 11, 11111, and 111111.*

The only reachable transition families not reproduced literally are two right-state matching families, which differ only by eager versus lazy debt discharge.

This gives the desired reverse implication of the research program: the original 2:1 architecture is not merely an example that happens to fit the theory; its control skeleton and transition arithmetic are recovered from the general construction.

13 Eager and lazy debt discharge

The historical machine and the centered 2:1 specialization use the same semantic debt but not always the same normalization schedule.

For example, on the reachable configuration $(R, 0)$ with input 0, the centered machine performs a matching pop and retains the right-state debt:

$$(R, 0) \xrightarrow{0}_{\text{lazy}} (R, \varepsilon). \quad (32)$$

The historical machine instead materializes the state debt immediately:

$$(R, 0) \xrightarrow{0}_{\text{eager}} (C, 111). \quad (33)$$

Both successors represent the same demand $(0, 3)$.

Likewise, for $k \geq 1$,

$$(R, 1^k) \xrightarrow{1}_{\text{lazy}} (R, 1^{k-1}), \quad (34)$$

$$(R, 1^k) \xrightarrow{1}_{\text{eager}} (C, 1^{k+2}). \quad (35)$$

The first leaves three virtual ones in state R ; the second writes them physically into the center stack.

Define the semantic canonical materialization map

$$\boxed{\mathcal{N}(c) = (C, \eta(\Theta(c)))}. \quad (36)$$

This is a semantic map, not a claim that every such materialization is one legal PDA transition.

Theorem 13.1 (Eager–lazy semantic equivalence). *Let $c_{\text{hist}}(w)$ and $c_{\text{cent}}(w)$ be the core configurations reached after the same binary prefix w by the historical 2:1 machine and the centered 2:1 specialization, respectively. Then*

$$\boxed{\Theta(c_{\text{hist}}(w)) = \Theta(c_{\text{cent}}(w)) = \mu(\rho_{2,1}(w))}. \quad (37)$$

Moreover, on the two reachable right-state matching families where the transition tables differ, the historical successor is exactly the canonical materialization of the centered successor:

$$\delta_{\text{hist}}(c, a) = \mathcal{N}(\delta_{\text{cent}}(c, a)).$$

Proof. Both machines satisfy the same residual update and the same shortest-demand projection, so the first identity follows from theorem 5.2 and the 2:1 residual invariant. For $(R, 0)$ on input 0, the centered successor is (R, ε) with

$$\Theta(R, \varepsilon) = V_R = (0, 3),$$

so $\mathcal{N}(R, \varepsilon) = (C, 111)$, the historical successor. For $(R, 1^k)$ on input 1, the centered successor $(R, 1^{k-1})$ has semantic demand $(0, k+2)$, whose canonical materialization is $(C, 1^{k+2})$, again the historical successor. $\square$

Thus the remaining difference after specialization is temporal rather than semantic:

$$\boxed{\text{historical 2:1} = \text{eager discharge}}, \quad \boxed{\text{centered 2:1} = \text{lazy discharge}}.$$

The general theory therefore recovers the architecture while also revealing a non-uniqueness of normalization chronology.

Corollary 13.2 (Historical machine as an eager refinement). *The historical 2:1 core is obtained from the centered 2:1 specialization by retaining every specialized centered transition except on the two reachable right-state matching families of theorem 13.1, where the centered lazy successor is replaced by its canonical materialization $\mathcal{N}$. Thus the return from the general theory to the motivating 2:1 design is exact at the level of control architecture, potentials, WRITE arithmetic, and shortest-demand semantics, and recovers the historical transition table after this eager normalization-schedule choice. Direct specialization alone is therefore not literal transition-table identity; the remaining distinction is precisely the eager-lazy scheduling choice.*

14 Computational audit

The proofs above are symbolic; computation is used only as an independent transcription and boundary-case check. A separate reference implementation of the centered transition schema performed breadth-first exploration for every ordered coprime pair

$$1 \leq p, q \leq 20$$

(including $(1, 1)$), a total of 255 ratios, through shortest reaching-word depth 80. No counterexample was found to:

1. transition totality on reached binary configurations;
2. the residual invariant of theorem 7.1, checked on every explored edge;
3. consistency of the residual attached to a configuration when that configuration is reached by more than one explored edge;
4. the two-cell visibility consequence of the canonical-segment invariant; or
5. $\rho = 0 \iff (C, \varepsilon)$ before the endmarker.

These tests are not premises of any theorem in the paper. Their role is to guard against sign, parity, and endpoint transcription errors. A compact audit script is distributed with this draft.

15 Related work and novelty boundary

Deterministic one-counter automata are classical [1], and modern work studies equivalence and reachability/shortest-path phenomena in one-counter systems [2, 3]. Thus neither signed counting nor shortest paths in counter-like systems are claimed as new here. Store configurations of language acceptors are themselves established mathematical objects [5].

The use of deterministic real-time pushdown graphs in shortest-path questions also has direct precedent. Carayol and Serre study deterministic real-time pushdown graphs whose outgoing edges are marked according to whether they lie on shortest paths to designated final vertices [4]. Our claim is not a new general shortest-path theory for pushdown graphs; it is the explicit family-specific identification of shortest completion demand with the state-stack semantics of the present normalization architecture.

Likewise, restrictions on pushdown alphabets and state/stack descriptive resources have a substantial literature, including stateless and real-time deterministic settings [6, 7]. The exact width theorem in theorem 11.5 is therefore stated only for the canonical deferred-SEARCH model of theorem 11.1. It is not an unrestricted state-complexity theorem for all DPDAs recognizing $L_{p,q}$.

Weighted pushdown automata are also an established general formalism [8]; our use of the word *weight* refers only to the elementary arithmetic coordinate $W(x, y) = px - qy$, not to a claim of introducing the weighted-PDA model.

The proposed contribution is narrower and structural: starting from one completion-aware 2:1 design, we isolate a primitive-kernel semantics, identify one-cell finite-control debt as the mechanism repairing the static LIFO visibility obstruction, construct a uniform real-time literal-demand normalizer for all coprime $p:q$, prove its two-cell qualitative visibility and exact WRITE normalization, obtain a matching deferred-model control width, and then recover the original $C/L/R$ architecture as the 2:1 specialization. The eager-lazy comparison further shows that the recovered semantics does not force a unique normalization schedule.

16 Conclusion

A completion-aware 2:1 DPDA suggested a mechanism that was not visible from recognition alone. Its stack represented unresolved completion needs; its side states carried finite corrections; and its cancellation rules operated on that demand online. Generalizing those ideas required identifying the true obstruction: a static two-visible stack ordering cannot survive both deletion directions under LIFO access if a displaced blocker is immediately restored.

The resulting repair is simple to state. Keep the literal demand on the stack, move a one-cell blocker into finite control as signed debt, and materialize that debt only when a WRITE boundary is reached. The primitive kernel $K = (q, p)$ governs the arithmetic of every such materialization. This yields a uniform real-time centered family for every coprime $p:q$, with exact shortest-demand projection and two-cell qualitative visibility.

The specialization back to 2:1 closes the conceptual loop. The general theory recovers $C/L/R$, the potentials $0, +3, -3$, the left cancellation behavior, the right locality-repair role, and the characteristic WRITE words. Direct centered specialization differs from the historical transition table only on two reachable right-state matching families; by theorem 13.2, choosing eager canonical materialization on exactly those families recovers the historical core. Thus the remaining distinction is one of normalization chronology rather than semantics. In this sense the 2:1 machine was not merely generalized: its unusual architecture is reconstructed from the general theory.

Statements and Declarations

Author contributions. Alp Eren Bütün: Conceptualization, methodology, formal analysis, investigation, software, validation, visualization, writing—original draft, and writing—review and editing.

Funding. No funding was received for conducting this study.

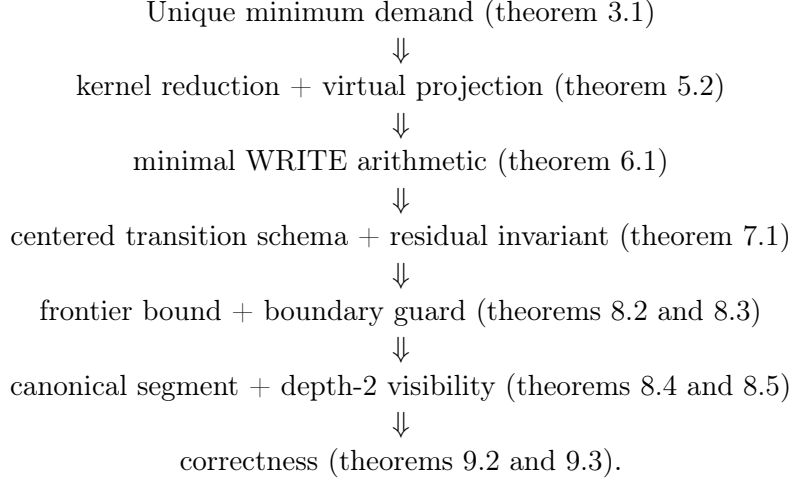
Competing interests. The author has no relevant financial or non-financial interests to disclose.

Data and code availability. The paper is theoretical and uses no empirical dataset. All definitions, constructions, and proofs needed for the mathematical results are contained in the manuscript. The reference audit script `centered_dpda_audit.py`, used for the computational consistency checks described in section 14, accompanies the manuscript source package. Related source-design and validation materials are publicly maintained in the author’s DPDA repository at <https://github.com/alperenbutun/dpda-ones-double-zeros>.

Ethics approval and consent. Not applicable.

A A compact theorem dependency map

For reading and proof auditing, the principal dependencies are:



The deferred-model lower bound in theorem 11.5 uses only the canonical-demand semantics and its separately stated model assumptions. The 2:1 recovery and eager-lazy theorems then specialize the general semantics.

References

- [1] L. G. Valiant and M. S. Paterson. Deterministic one-counter automata. *Journal of Computer and System Sciences*, 10(3):340–350, 1975.
- [2] S. Böhm, S. Göller, and P. Jančár. Equivalence of deterministic one-counter automata is NL-complete. arXiv:1301.2181, 2013.
- [3] D. Chistikov, W. Czerwiński, P. Hofman, M. Pilipczuk, and M. Wehar. Shortest paths in one-counter systems. *Logical Methods in Computer Science*, 15(1):19:1–19:28, 2019.
- [4] A. Carayol and O. Serre. Marking shortest paths on pushdown graphs does not preserve MSO decidability. *Information Processing Letters*, 116(10):638–643, 2016.
- [5] O. H. Ibarra and I. McQuillan. On store languages of language acceptors. *Theoretical Computer Science*, 745:114–132, 2018.
- [6] T. Masopust. A note on limited pushdown alphabets in stateless deterministic pushdown automata. *International Journal of Foundations of Computer Science*, 24(3):319–328, 2013.
- [7] G. Pighizzini. Deterministic pushdown automata and unary languages. *International Journal of Foundations of Computer Science*, 20(4):629–645, 2009.
- [8] A. Butoi, B. DuSell, T. Vieira, R. Cotterell, and D. Chiang. Algorithms for weighted pushdown automata. In *Proceedings of EMNLP 2022*, pages 9669–9680, 2022.
- [9] W. D. Neumann and M. Shapiro. Regular geodesic normal forms in virtually abelian groups. *Bulletin of the Australian Mathematical Society*, 55(3):517–519, 1997.